

The Forge

Balance Sandbox — a proactive design sample for SEED

Game Systems & Mechanics Designer · Aliaskar Bekishev · alyasska.github.io · github.com/Alyasska/seed-artifact · lyasskar@gmail.com

An **in-silico playtest** for one Forge system — territory control, the bridge momentum layer, player classes, and a siege endgame. You can't cheaply playtest a 500-person, 90-minute field game, so this models one system and runs thousands of matches headless to read the balance *before* anyone books a venue. The same rules engine drives the watchable live match **and** the headless sims — a designer/GM pre-flight rig, not a demo game. It is the deliverable the role names: a **living design document**, **balance logic**, and **playtest frameworks** — not just code.



Live match on a real DEM of Lake San Antonio (a Lightning-in-a-Bottle site). The reservoir & creeks become the bridge-gated ravine; travel-time follows real slope. Gold spans are built bridges, dashed lines unbuilt crossings, diamonds are strongholds (with Wall arcs), dots are class-typed squads.

What it found

I. SELF-PLAY FOUND AN EXPLOIT THE HEURISTICS DIDN'T

A policy trained from a blank genome reaches **~100%** against the four hand-written bots — it surfaced a hole a scripted pool would have called "balanced." RL here is a *better playtester*, not flashy AI. (Cross-Entropy Method, no GPU; a gym-microrrts/PPO skeleton ships for the scaled version.)

II. TERRAIN FLIPS BALANCE

The same ruleset that's fine on a symmetric board goes **aggressor-dominant (~75%)** on real Lake San Antonio terrain, and the venue runs side-biased. Balance is *per-venue*, not one-and-done — that's the recurring service.

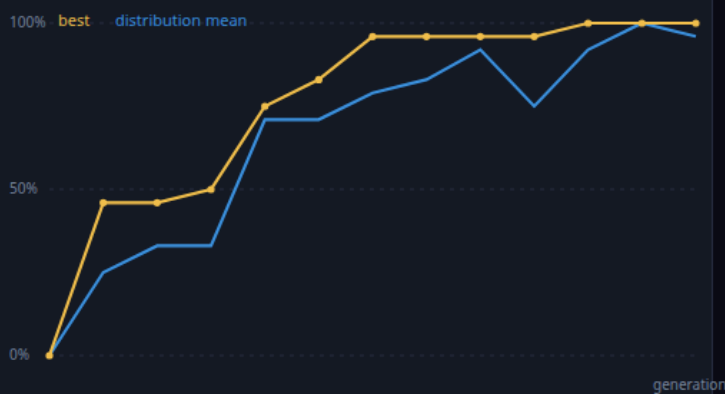
III. POLICIES DON'T FULLY TRANSFER · PURE TURTLING IS NON-VIABLE

The board-trained policy slips **~85% → ~70s%** on real terrain (a transfer gap), and turtling can't cross the ravine. Both surface as design calls, each with the levers to change them.

The Forge — self-play playtester (CEM), learning curve

From a **blank genome** (gen 0 = naive play) to **100% win-rate** over 12 generations — learned from scratch, no GPU, dependency-free (CEM).
PPO/gymnasium skeleton: `rl/forgenv.py`.

Win-rate vs the heuristic pool (expander · aggressor · bridger · turtle)



What it discovered (learned weights)

proximity	1.44
neutral	1.50
enemyOwned	-0.43
ownOwned	-2.01
enemyStronghold	0.01
siegeTowerP4	2.84
contested	0.52
homeProximity	-0.95
isTower	0.16

Positive = the policy seeks it. The win comes from weighting the **Phase-4 siege tower** and forward pressure, and down-weighting sitting on owned ground — i.e. it taught itself to rush the siege, the same exploit the Monte-Carlo flags.

Self-play learning curve: blank genome → ~100% win-rate vs the heuristic pool over 12 generations, with the weights it discovered (it taught itself to rush the Phase-4 siege tower).

How it works

levers 10 balance dials, each with an **honesty ledger** — intent / trade-off / failure mode.

playtest Monte Carlo over all strategy matchups: faction fairness, dominant-strategy detection, snowball vs comeback, dead-time, siege rate, matchup matrix.

mechanics Presence-by-pulse capture (NFC-tap-shaped events), classes, the 90-minute phase arc, walls + battering + delay towers, fog-of-war intel.

stack Plain ES-module JS (no build, runs on Pages) · Python+Pillow for the DEM pull · seeded & reproducible (`node test/smoke.mjs`).

Levers

Honesty ledger

Playtest

You can't cheaply playtest 500 people on a field. So play it in silico — same rules engine, thousands of matches, balance read off the aggregate.

Matches

400

▼

▶ Run playtest at current levers

400 matches · 5² matchups · 0ms · seeded

✓ Side-neutral: Red 45% / Blue 55% — fair map.

⚠ aggressor leads at 76% — aggressive openings favored; nerf candidates: capRate ↓, decay ↑.

◆ Self-play RL: 74% (trained on the symmetric board — the transfer gap to this terrain is itself a finding).

◆ Pure turtling is non-viable (11%) — it can't cross the ravine. Design call: give defense a path, or accept it.

Siege fires in 62% of games, wins 9%.

Lead persistence 85% / comeback 15% — raise comeback aid to claw it back.

Win rate by strategy

Expander

46%

Aggressor

76%

Bridger

43%

Turtle

11%

RI

74%

Faction fairness

Red

45%

Blue

55%

Engagement

85% lead persists

15% comeback

1.8 lead changes

11% dead-time

62% siege fires

Matchup matrix

	Expa	Aggr	Brid	Turt	RI
Expa	56	6	56	100	31
Aggr	94	56	100	88	25
Brid	75	0	19	100	6
Turt	0	0	0	0	0
RI	69	44	94	100	0

Proof artifact for SEED · The Forge — Game Systems & Mechanics Designer.

Aliaskar Bekishev · [github.com/Alyasska](#) · [alyasska.github.io](#)

The balance readout on the real field — findings, win-rate by strategy, and the matchup matrix.

End vision. The pre-flight for The Forge: tune economy, class weights, and phase timings **per venue** in sim → ship a near-balanced rulebook → spend the few expensive on-field pilots validating how real humans behave, not discovering that turtling is broken with 500 people on a field.

Scope, honestly: one system, finished small. Classes / intel / parley are designed in full but only partly wired; bots map the structural balance envelope, not the feel.